AIops Assignment 1 (DA3408)

Abhijeet Kumar(DA24B001)

Q1. Technical Debt Diagnosis

(a) Change in rounding logic for "estimated delivery time" affected "favorite restaurants" recommendations

I would classify this as **Entanglement (CACE)**. At first, these two features seem completely unrelated, so changing the delivery-time calculation should not affect the favorite-restaurants recommendation. However, in an ML model, different features are connected through the parameters learned during training. Even a small change in the distribution of one feature can therefore affect the relationships learned by the model and ultimately change the predictions for other features. This is an example of the "Changing Anything Changes Everything" problem in ML systems.

(b) Marketing dashboard silently using the model's raw output table

This is an example of **Undeclared Consumers**. The marketing team is depending on the model's output table, but the ML team does not know about this dependency. Because of this, the ML team may change the table structure, retrain the model, or remove the table without realizing that it will affect another team. The main problem is that the dependency is hidden and has not been formally documented.

(c) Training pipeline consisting of 14 undocumented shell scripts

I would classify this as **Configuration & Glue-Code Debt**. The pipeline does not have a proper orchestration system, and its logic is spread across many undocumented scripts. This makes the pipeline difficult to understand, reproduce, maintain, or hand over to another person. It can work for some time, but it becomes fragile because there is no clear single source of truth for how the complete training process should be executed.

2. Proposed mitigation

I would choose to address **(b) Undeclared Consumers**, because this problem can be reduced by making the model and its outputs properly managed and documented.

Currently, the marketing team is directly accessing the raw output table without the ML team's knowledge. A better approach would be to manage the model through the **MLflow Model Registry** and clearly document its output schema and consumers. The model can be registered and its production/staging versions can be managed through MLflow instead of allowing teams to depend directly on an undocumented raw table.

I would also maintain information about who is consuming the model output and for what purpose using documentation and MLflow metadata such as tags or notes. This would

make the dependency visible to the ML team. As a result, before changing the model or its output format, the team can identify the affected consumers and check whether the change will break their applications. This is much safer than discovering a hidden dependency only after something breaks.

Q2: MLflow Experiment Comparison

In this experiment, I used MLflow to track and compare different runs of an MLP classifier on the MNIST dataset. The main idea was to see how changing the model architecture, learning rate, and batch size affects the final performance. I ran six different combinations of these parameters and logged the parameters and metrics of every run in MLflow.
The six configurations and their validation performance are summarized below.

Table 1: MLP Hyperparameter Configurations and Results

Run	Hidden Layers	LR	Batch Size	Val. Acc.	Val. F1
1	(64,)	0.001	32	0.9410	0.940
2	(64,)	0.01	32	0.9370	0.936
3	(128,)	0.001	32	**0.9515**	**0.951**
4	(128,)	0.01	32	0.9355	0.935
5	(128, 64)	0.001	64	0.9490	0.949
6	(128, 64)	0.01	64	0.9020	0.904

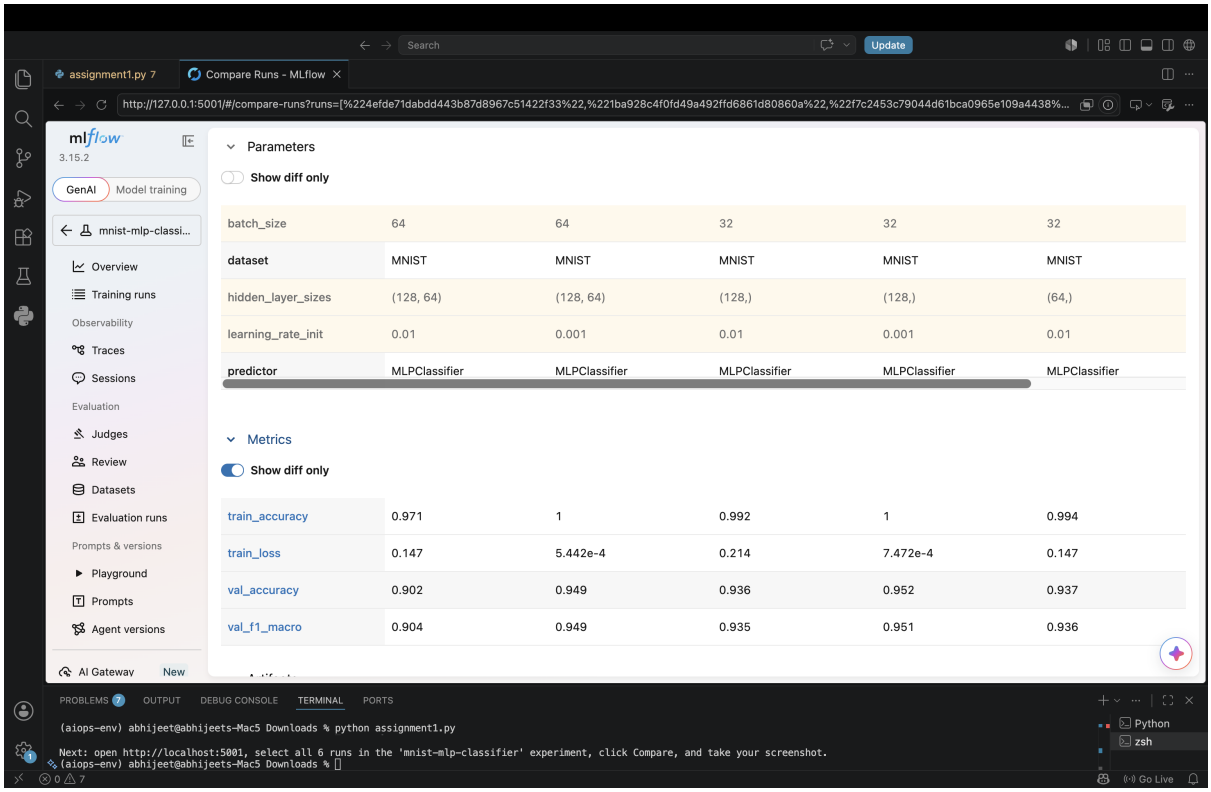

Figure 1: MLflow comparison table showing parameters and metrics for all six MLP runs.

My observations and analysis

Six MLP experiments were conducted on a 10,000-sample subset of MNIST while varying the hidden-layer architecture, learning rate, and batch size. The best-performing configuration was **MLP (128,) with a learning rate of 0.001 and batch size 32**, achieving a validation accuracy of **0.9515** and a macro F1-score of approximately **0.951**. This configuration performed better than the smaller (64,) network and the deeper (128, 64) architecture under the tested settings as with this setting i guess it learned steadily.

There is evidence of **overfitting** in the experiments. The best run achieved a training accuracy of **1.0000** with a very low training loss of **0.000747**, while its validation accuracy was only **0.9515**. Thus, the model fits the training data almost perfectly but does not achieve equivalent performance on unseen data. Similar behavior is visible in the other low-learning-rate configurations, where training accuracy reaches 1.0 while validation accuracy remains below 0.95.

Among the tested hyperparameters, the **learning rate appears to have the largest effect on performance**. Reducing the learning rate from 0.01 to 0.001 consistently improved validation accuracy. For example, the (128, 64) architecture improved from **0.9020 to 0.9490** when the learning rate was reduced. Therefore, **0.001 provided substantially better generalization than 0.01** in these experiments.

MLflow Parameter Logging

The following parameters were explicitly logged for every run:

```
mlflow.log_param("hidden_layer_sizes", str(hidden_layer_sizes))
mlflow.log_param("learning_rate_init", learning_rate_init)
mlflow.log_param("batch_size", batch_size)
mlflow.log_param("predictor", "MLPClassifier")
mlflow.log_param("dataset", "MNIST")
```

MLflow Metric Logging

The following metrics were calculated and logged for each run:

```
mlflow.log_metric("train_accuracy", train_acc)
mlflow.log_metric("val_accuracy", val_acc)
mlflow.log_metric("val_f1_macro", val_f1)
mlflow.log_metric("train_loss", train_loss)
```

Best Run

The best run was identified programmatically using MLflow by sorting the runs according to validation accuracy in descending order.

```
runs_df = mlflow.search_runs(
```

```
    experiment_names=["mnist-mlp-classifier"],
    order_by=["metrics.val_accuracy DESC"],
)

best_run = runs_df.iloc[0]
```

The best configuration was:

- **Hidden layer architecture:** (128,)
- **Learning rate:** 0.001
- **Batch size:** 32
- **Training accuracy:** 1.0000
- **Validation accuracy:** 0.9515
- **Training loss:** 0.000747
- **Validation macro F1:** approximately 0.951

Q3. DVC Data Versioning and Rollback

I used DVC to version the dataset and demonstrate rollback between two versions.

Version 1: The initial dataset contained 1800 files. After creating `file_list.csv`, it contained 1801 lines including the header. I tracked it using DVC, committed the metadata to Git, and tagged it as `v1`.

Version 2: I added the files from `new-labels.zip`, increasing the dataset to 2800 files. The updated `file_list.csv` contained 2801 lines. I tracked the change using DVC and tagged it as `v2`.

Rollback: To test versioning, I checked out `v1` and ran `dvc checkout`. The file count changed from 2801 lines to 1801 lines, confirming that DVC successfully restored the earlier version.

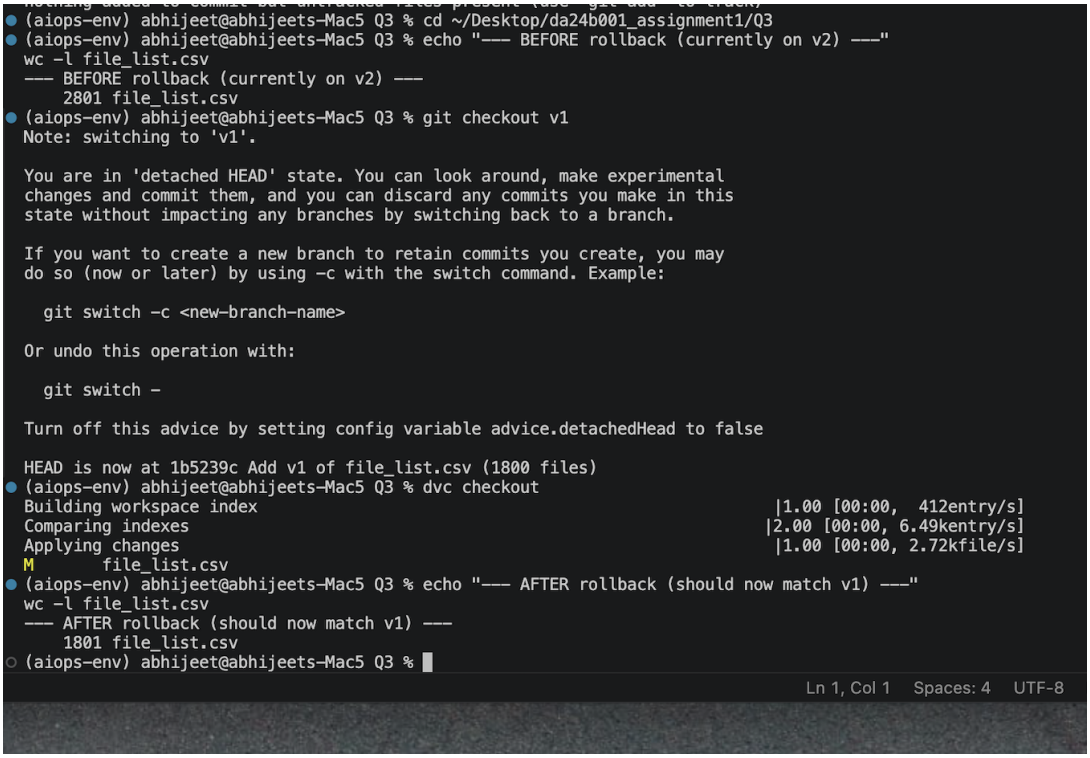

Figure 2: DVC rollback from v2 to v1.

Q4

I am the partner B and after reproducing the mlflow pipeline. I verified the metrices values and the results.